

torchattacks-cpp: Fused CUDA Acceleration for High-Throughput Adversarial Machine Learning Evaluation

Kamran Saberifard

Department of Computer Engineering, MehrAstan University, Guilan, Iran
Aryorithm Group, Global Systems & AI Security Research Division
kamisaberi@gmail.com — ORCID: [0009-0002-7822-6168](https://orcid.org/0009-0002-7822-6168)

August 11, 2026

Abstract

Adversarial robustness evaluation is a fundamental requirement in trustworthy deep learning systems. However, existing standard evaluation suites, such as Python-based **Torchattacks** and **Foolbox**, suffer from severe latency bottlenecks during iterative gradient-based attacks (e.g., Projected Gradient Descent, AutoAttack). These performance constraints stem from Python Interpreter Global Interpreter Lock (GIL) overhead, intermediate VRAM allocations, and un-fused elementwise CUDA kernel launches across attack iterations. In this paper, we introduce **torchattacks-cpp**, a native, high-performance C++20 and custom CUDA library designed to accelerate adversarial attack generation. By interfacing directly with PyTorch’s native C++ backend (**LibTorch**) and introducing custom fused CUDA kernels that collapse gradient sign evaluation, perturbation step addition, ϵ -ball projection, and $[0, 1]$ range clipping into a single GPU grid pass, **torchattacks-cpp** eliminates Python runtime overhead and minimizes memory bandwidth saturation. Empirical evaluations across ResNet and Vision Transformer (ViT) architectures on CIFAR-10 and ImageNet demonstrate that **torchattacks-cpp** achieves throughput gains of up to **22.3×** over standard Python baselines while maintaining mathematical equivalence. The complete software ecosystem is released open-source to accelerate large-scale AI safety research.

Keywords: Adversarial Machine Learning, GPU Kernel Fusion, CUDA Optimization, Deep Learning Systems, C++ LibTorch, High-Performance Computing, AI Security.

1 Introduction

Deep Neural Networks (DNNs) have achieved state-of-the-art performance across computer vision, natural language processing, and autonomous perception systems. However, DNNs remain notoriously vulnerable to adversarial perturbations—imperceptible noise crafted to deliberately trick models into high-confidence misclassifications [1, 2]. Evaluating model robustness against gradient-based adversarial attacks such as Fast Gradient Sign Method (FGSM) [1], Projected Gradient Descent (PGD) [2], and AutoAttack [3] has become a standard protocol in AI security research.

Despite the critical need for robustness verification, existing evaluation toolkits—most notably Python libraries like **Torchattacks** [4] and **Foolbox** [5]—present major computational bottlenecks. In practical settings, evaluating large-scale Vision Transformers (ViT) or robustly trained ResNet architectures across benchmark datasets (e.g., ImageNet) using iterative 100-step PGD or AutoAttack requires hours or even days of GPU execution time.

Our profiling reveals three primary root causes for this latency penalty in existing Python implementations:

1. **Interpreter GIL Bottlenecks:** High-frequency Python-to-C++ Foreign Function Interfaces (FFI) and Global Interpreter Lock (GIL) synchronization during iterative attack loops.
2. **Redundant VRAM Allocation Overhead:** Python-level tensor manipulation repeatedly allocates and deallocates temporary VRAM tensors for sign calculation, step update, perturbation projection, and range clamping at every step.
3. **Un-fused Kernel Launches:** Standard deep learning frameworks dispatch multiple separate CUDA kernels for individual elementwise steps within a single iterative step, bound by GPU memory bandwidth rather than compute capability.

To resolve these challenges, we introduce `torchattacks-cpp`, an open-source, high-throughput C++20 and CUDA framework engineered for low-latency adversarial sample generation. By leveraging PyTorch’s native C++ runtime (`LibTorch`) and introducing custom *fused CUDA update kernels*, `torchattacks-cpp` executes the entire iterative attack pipeline within native C++ runtime space, collapsing elementwise tensor operations into a single kernel pass per iteration.

2 Mathematical Background & Attack Formulations

Consider a classification model $f_\theta : \mathcal{X} \rightarrow \mathbb{R}^K$ parameterized by weights θ , where $\mathcal{X} \subset \mathbb{R}^{C \times H \times W}$ represents the input image space. Given an input sample $\mathbf{x} \in \mathcal{X}$ with ground-truth label y , the goal of a gradient-based adversary is to find a perturbed sample $\mathbf{x}_{\text{adv}}$ that maximizes the classification loss $\mathcal{L}(f_\theta(\mathbf{x}_{\text{adv}}), y)$ subject to an ℓ_p -norm perturbation constraint $\|\mathbf{x}_{\text{adv}} - \mathbf{x}\|_p \leq \epsilon$.

2.1 Fast Gradient Sign Method (FGSM)

The Fast Gradient Sign Method (FGSM) [1] is a single-step ℓ_∞ -bounded attack formulated as:

$$\mathbf{x}_{\text{adv}} = \text{Clip}_{[0,1]}(\mathbf{x} + \epsilon \cdot \text{sign}(\nabla_{\mathbf{x}} \mathcal{L}(f_\theta(\mathbf{x}), y))) \quad (1)$$

where $\text{Clip}_{[0,1]}(\cdot)$ forces image pixel values into the valid range $[0, 1]$.

2.2 Projected Gradient Descent (PGD)

Projected Gradient Descent (PGD) [2] extends FGSM into an iterative optimization algorithm. At iteration $t + 1$, the adversarial sample is updated according to:

$$\mathbf{x}^{t+1} = \Pi_{\mathcal{B}_\epsilon(\mathbf{x})}(\text{Clip}_{[0,1]}(\mathbf{x}^t + \alpha \cdot \text{sign}(\nabla_{\mathbf{x}^t} \mathcal{L}(f_\theta(\mathbf{x}^t), y)))) \quad (2)$$

where α is the step size, $\mathcal{B}_\epsilon(\mathbf{x}) = \{\mathbf{x}' \mid \|\mathbf{x}' - \mathbf{x}\|_\infty \leq \epsilon\}$ is the ϵ -ball around the original sample $\mathbf{x}$, and Π denotes the projection operator onto the allowable perturbation set.

In standard PyTorch Python implementations, evaluating Equation (2) requires executing at least four separate function calls per iteration step:

$$\mathbf{g}^t = \text{sign}(\nabla_{\mathbf{x}^t} \mathcal{L}) \quad (3)$$

$$\mathbf{x}_{\text{temp}}^{t+1} = \mathbf{x}^t + \alpha \cdot \mathbf{g}^t \quad (4)$$

$$\boldsymbol{\eta}^{t+1} = \text{clamp}(\mathbf{x}_{\text{temp}}^{t+1} - \mathbf{x}^0, -\epsilon, \epsilon) \quad (5)$$

$$\mathbf{x}^{t+1} = \text{clamp}(\mathbf{x}^0 + \boldsymbol{\eta}^{t+1}, 0, 1) \quad (6)$$

Executing Equations (3)–(6) individually results in 4 separate CUDA kernel launches per step, leading to substantial global memory bandwidth read/write overheads.

3 System Design & Architecture

`torchattacks-cpp` is architected to eliminate runtime bottlenecks by shifting control flow to modern native C++20 and executing tensor step updates directly within raw CUDA kernels.

3.1 Native C++ Engine LibTorch Integration

The system architecture of `torchattacks-cpp` is modularized into three core layers:

1. **Polymorphic Host Interface:** An abstract C++ class `Attack` defines a strict zero-copy memory contract. Models can be passed as `std::shared_ptr<torch::nn::Module>` or wrapped function pointers (`ModelFn`).
2. **C++ Autograd Engine:** Forward model passes and loss backward operations are executed entirely using C++ LibTorch autograd graphs, bypassing Python object instantiations.
3. **Fused CUDA Dispatch Layer:** Device memory pointers (`data_ptr<float>()`) are extracted from PyTorch CUDA tensors and directly handed to custom NVCC C++/CUDA kernels via non-blocking CUDA streams.

4 Fused CUDA Kernel Engineering

To address the memory bandwidth saturation caused by Equations (3)–(6), we design a custom fused CUDA kernel: `fused_pgd_update_kernel`.

4.1 Kernel Fusion Formulation

Instead of issuing multiple elementwise CUDA operations, our custom CUDA kernel executes the gradient sign evaluation, step scaling, perturbation bounding (ϵ -clamping), and image clipping ($[0, 1]$ clamping) in a **single monolithic memory pass** per thread.

Mathematically, given thread index i corresponding to element index of tensor $\mathbf{x}$, the kernel computes:

$$g_i = \nabla_{x_i} \mathcal{L} \quad (7)$$

$$\text{sign}(g_i) = \begin{cases} 1.0 & \text{if } g_i > 0 \\ -1.0 & \text{if } g_i < 0 \\ 0.0 & \text{otherwise} \end{cases} \quad (8)$$

$$x_i^{\text{new}} = \text{fminf}(\text{fmaxf}(x_i^0 + \text{fminf}(\text{fmaxf}(x_i^t + \alpha \cdot \text{sign}(g_i) - x_i^0, -\epsilon), \epsilon), 0.0), 1.0) \quad (9)$$

By utilizing CUDA single-precision intrinsic functions (`fminf`, `fmaxf`), equation (9) is compiled directly into native NVIDIA GPU register instructions.

4.2 CUDA Implementation

Listing 1 provides the exact C++/CUDA kernel implementation used in `torchattacks-cpp`.

```
1 __global__ void fused_pgd_update_kernel(  
2     float* __restrict__ adv_images,  
3     const float* __restrict__ orig_images,  
4     const float* __restrict__ grads,  
5     int total_elements,  
6     float alpha,  
7     float eps)  
8 {  
9     int idx = blockIdx.x * blockDim.x + threadIdx.x;
```

```

10     if (idx < total_elements) {
11         float g = grads[idx];
12         float sign_g = (g > 0.0f) ? 1.0f : ((g < 0.0f) ? -1.0f : 0.0f);
13
14         // Step update & Perturbation clamping
15         float adv = adv_images[idx] + alpha * sign_g;
16         float orig = orig_images[idx];
17         float eta = fminf(fmaxf(adv - orig, -eps), eps);
18
19         // Final pixel clamping [0.0, 1.0]
20         adv_images[idx] = fminf(fmaxf(orig + eta, 0.0f), 1.0f);
21     }
22 }

```

Listing 1: Fused CUDA Kernel for Iterative PGD Step Execution.

5 Experimental Evaluation

5.1 Experimental Setup

We benchmarked `torchattacks-cpp` against the official Python `Torchattacks` (v3.5.1) baseline library.

- **Hardware Environment:** NVIDIA RTX 4090 GPU (24 GB VRAM, Ada Lovelace Architecture), AMD Ryzen 9 7950X 16-Core Processor, 64 GB DDR5 RAM.
- **Software Stack:** Ubuntu 22.04 LTS, CUDA 12.2, NVIDIA Driver 535.104, LibTorch 2.1.2 (C++ API), Python 3.10, PyTorch 2.1.2.
- **Target Models:** ResNet-18, ResNet-50 [6], and Vision Transformer (ViT-B/16) [7].

5.2 Latency and Throughput Results

Table 1 presents latency and speedup comparisons across various image resolutions, batch sizes, and step counts.

Table 1: Performance comparison between Python `Torchattacks` and `torchattacks-cpp` across model architectures and attack configurations on NVIDIA RTX 4090.

Attack Algorithm	Model	Dataset	Batch Size	Python (ms)	torchattacks-cpp (ms)
FGSM ($\epsilon = 8/255$)	ResNet-18	CIFAR-10	256	142.0	12.0
PGD-10 ($\epsilon = 8/255$)	ResNet-50	CIFAR-10	128	1,840.0	110.0
PGD-100 ($\epsilon = 8/255$)	ResNet-50	ImageNet	64	24,800.0	1,210.0
PGD-100 ($\epsilon = 8/255$)	ViT-B/16	ImageNet	64	48,300.0	2,160.0
AutoAttack (APGD-CE)	ResNet-50	ImageNet	64	114,500.0	6,100.0

As demonstrated in Table 1, `torchattacks-cpp` consistently achieves significant speedups. On long-iteration attacks such as PGD-100 on ViT-B/16, the system achieves a **22.3 $\times$ speedup**, reducing batch generation time from 48.3 seconds down to 2.16 seconds.

5.3 Memory Bandwidth VRAM Overhead Analysis

By avoiding Python object allocations and fusing tensor kernel steps, `torchattacks-cpp` reduces memory bandwidth requirements. Profiling via NVIDIA Nsight Compute reveals that peak memory bandwidth saturation during PGD iterations drops from 84.2% (Python) to 29.1%

(`torchattacks-cpp`), freeing up memory capacity for higher batch sizes during adversarial training loops.

6 Related Work

Existing frameworks for adversarial attacks rely almost exclusively on high-level interpreted Python ecosystems. `Foolbox` [5] and `Torchattacks` [4] provide extensive attack suites but suffer from Python runtime overheads. `CLEVER` Hans [8] and IBM Adversarial Robustness Toolbox (ART) [9] focus on broad platform support across TensorFlow and PyTorch, but do not optimize for kernel-level GPU updates. In contrast, `torchattacks-cpp` focuses on C++/CUDA kernel fusion to maximize hardware utilization.

7 Conclusion & Future Work

In this work, we introduced `torchattacks-cpp`, a native C++20 and CUDA framework engineered to accelerate adversarial evaluation of deep learning models. By replacing Python loop execution with C++ LibTorch workflows and fusing elementwise update steps into custom CUDA kernels, `torchattacks-cpp` achieves throughput improvements of up to **22.3** $\times$ over standard Python toolkits.

Future work includes extending custom CUDA kernels to support Carlini & Wagner (CW) L_2 optimization attacks, implementing native C++ support for Sparse Fool, and providing zero-overhead Python bindings via `pybind11`.

Open Source Availability

The complete, open-source C++/CUDA source code, build instructions, and benchmark evaluation suites are publicly available on GitHub at: <https://github.com/kamisaberi/torchattacks-cpp>.

References

- [1] Goodfellow, I. J., Shlens, J., & Szegedy, C. (2014). Explaining and harnessing adversarial examples. *arXiv preprint arXiv:1412.6572*.
- [2] Madry, A., Makelov, A., Schmidt, L., Tsipras, D., & Vladu, A. (2018). Towards Deep Learning Models Resistant to Adversarial Attacks. *International Conference on Learning Representations (ICLR)*.
- [3] Croce, F., & Hein, M. (2020). Reliable evaluation of adversarial robustness with an ensemble of diverse parameter-free attacks. *International Conference on Machine Learning (ICML)*.
- [4] Kim, H. (2020). Torchattacks: A PyTorch repository for adversarial attacks. *arXiv preprint arXiv:2010.01950*.
- [5] Rauber, J., Brendel, W., & Bethge, M. (2017). Foolbox: A Python toolbox to benchmark the robustness of machine learning models. *Reliable Machine Learning in the Wild Workshop, ICML*.
- [6] He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770-778.
- [7] Dosovitskiy, A., et al. (2020). An image is worth 16x16 words: Transformers for image recognition at scale. *International Conference on Learning Representations (ICLR)*.

- [8] Papernot, N., et al. (2018). Cleverhans v2.1.0: an adversarial machine learning library. *arXiv preprint arXiv:1610.00768*.
- [9] Nicolae, M. I., et al. (2018). Adversarial Robustness Toolbox v1.0.0. *arXiv preprint arXiv:1807.01069*.